

PROJECT DESIGN REPORT (PDR)

Project Title: Automated Sentiment Intelligence Engine (ASIE)

Author: Sharjeel Ansari

Date: January 2024

Category: MLOps / AI Infrastructure / Distributed Systems

1. EXECUTIVE SUMMARY

The **Automated Sentiment Intelligence Engine (ASIE)** is a production-grade MLOps platform designed to solve the "Static Model" problem in Machine Learning. While traditional DevOps ensures code reliability, MLOps must ensure *model* reliability as real-world data shifts. This project implements a **Level 2 MLOps Maturity** architecture—a self-healing, closed-loop system that monitors for statistical data drift, triggers automated retraining on GPU-accelerated cloud infrastructure, and redeploys optimized models without manual intervention.

2. PROBLEM STATEMENT & OBJECTIVE

2.1 The Problem

Machine Learning models suffer from **Model Decay**. A sentiment analysis model trained on formal financial news often fails when exposed to the evolving slang of retail investment forums (e.g., Reddit's r/WallStreetBets). Standard DevOps health checks (CPU/RAM) remain "Green" while the model's business value drops to zero because it no longer understands the input data.

2.2 Objective

- **Bridge the Gap:** Transition from manual model deployment to automated lifecycle management.
- **Achieve Zero-Downtime Retraining:** Automatically update models when performance drops below a 0.80 F1-score.
- **Infrastructure Efficiency:** Use transient AWS Spot instances to reduce GPU training costs by ~70%.

3. DATA ARCHITECTURE

3.1 Baseline Dataset (Training)

- **Source:** Financial PhraseBank (sentences_allagree).
- **Volume:** ~2,300 expert-labeled financial news sentences.
- **Storage:** AWS S3, versioned and tracked via **DVC (Data Version Control)**.

3.2 Live Stream (Inference)

- **Source:** Real-time headlines via **NewsAPI**.
- **Ingestion:** Python-based ingestion engine running as a CronJob on **AWS EKS**.

3.3 Synthetic Drift (Validation)

- **Method:** Injection of Reddit/Twitter-style "Slang" data to trigger statistical anomalies and test the self-healing capabilities of the monitor.

4. TECHNICAL SPECIFICATIONS & TOOLS

Layer	Technology	Justification
Orchestration	Kubernetes (AWS EKS)	Provides scalable, containerized environments for inference and monitoring.
Model Registry	MLflow	Centralized management of model versions, metrics, and hyperparameter logs.
Data Tracking	DVC	Ensures 100% reproducibility by linking Git commits to specific S3 data snapshots.
Model Serving	FastAPI	Selected for high-concurrency, low-latency asynchronous processing.
GPU Optimization	NVIDIA CUDA/cuDNN	Packaged via Docker to ensure environment parity between local and cloud.
Monitoring	Evidently AI	Specialized in detecting Kolmogorov-Smirnov (KS) data drift and target drift.

5. SYSTEM ARCHITECTURE (THE CLOSED LOOP)

1. **Prediction:** The FastAPI service on EKS serves the DistilBERT model.
2. **Observation:** Each request is sampled and sent to a "Shadow Bucket" in S3.
3. **Analysis:** An **Evidently AI** service compares the "Shadow Bucket" against the "DVC Baseline."
4. **Trigger:** If the drift score exceeds 0.5, a webhook fires to **Apache Airflow**.
5. **Execution:** Airflow provisions a transient **EC2 G4dn** instance, fetches the new data, retrains the model, and pushes the new version to **MLflow**.
6. **Deployment:** **ArgoCD** detects the model update and performs a rolling update of the EKS pods.

6. IMPLEMENTATION ROADMAP (6 WEEKS)

- **Week 1:** Initialize DVC, MLflow, and baseline training on [financial_phrasebank](#).
- **Week 2:** Containerize inference using **FastAPI** and **NVIDIA-Docker**.
- **Week 3:** Provision **AWS EKS** and deploy the inference service with Helm.
- **Week 4:** Build the **Airflow** DAG for automated model training.
- **Week 5:** Implement **Evidently AI** drift monitoring and Prometheus alerting.
- **Week 6:** Perform the "Drift Stress Test" and finalize the self-healing logic.

7. RISK MITIGATION & SECURITY

- **Risk:** Data Poisoning during automated retraining.
 - *Mitigation:* Implement a "Human-in-the-Loop" gateway requiring a manual Slack approval before a new model is tagged "Production."
- **Risk:** GPU Out-of-Memory (OOM) errors.
 - *Mitigation:* Implement dynamic batching and mixed-precision (FP16) training.
- **Security:** Unauthorized access to training data.
 - *Mitigation:* Use **AWS IAM Roles for Service Accounts (IRSA)** to provide the EKS pods with least-privilege access to S3.